

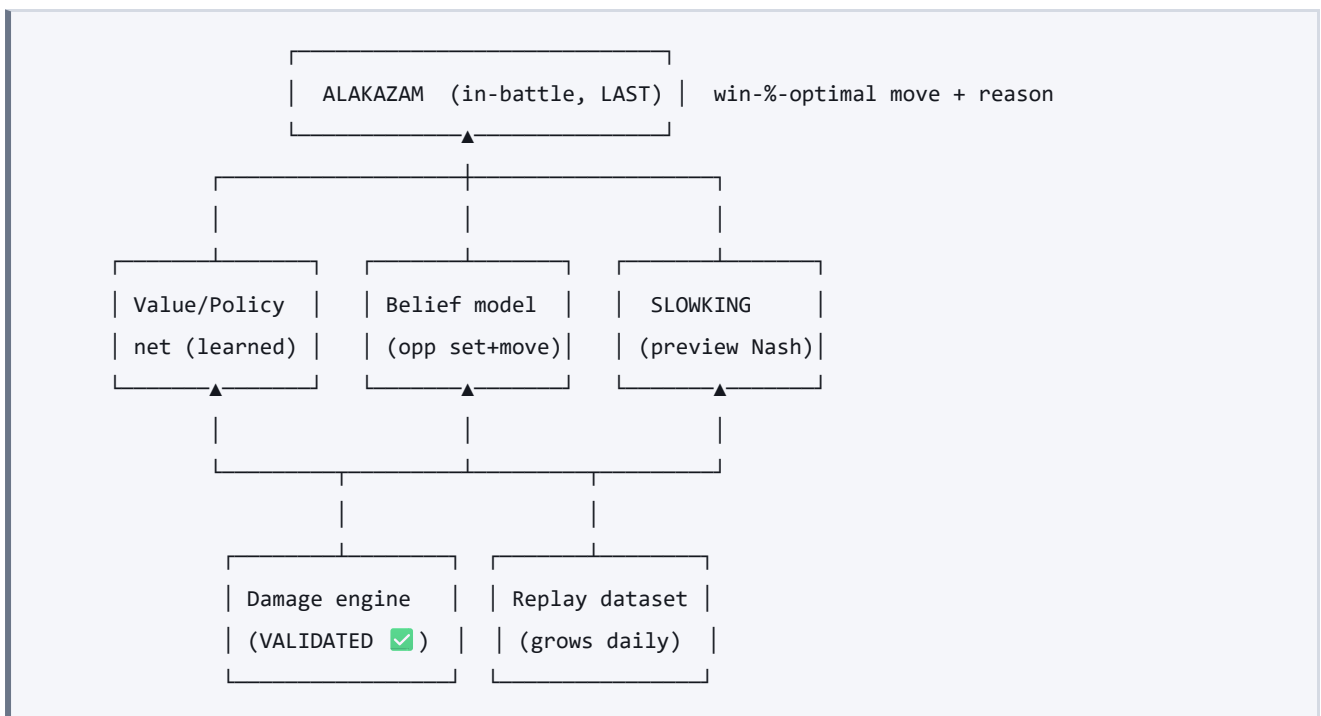
ALAKAZAM v2 — Architecture Spec (the capstone, built last)

Turns `docs/LITERATURE-v2.md` into a concrete, phased build. **ALAKAZAM is the final boss and is built LAST**, because it consumes the other models as inputs. We secure each input — with an honest acceptance bar (proper-score metric + CI + baseline, persisted to JSON, run in CI) — before assembling the capstone.

Guiding principles (from the study):

- **Decisions, not outcomes.** Never predict the match winner; output expected value of decisions.
- **Learn from human data first** (Metamon), **KL-anchor to human play** (CICERO), **mix strategies** (regret matching / R-NaD), **Nash-select for robustness** (AlphaStar/PSRO), **search only to refine** (ReBeL/DeepStack). Present as calibrated EV (sports xG/EPV).
- **Build on the one validated thing** (exact damage). Everything ships with a CI + baseline.

The dependency graph



CHOMP (pivoted) sits alongside SLOWKING (bring/lead EV) and feeds the belief ("what did they bring").

Input models — secure these first (each with an acceptance bar)

Io. Damage engine — ✓ SECURED

- **Is:** validated Gen-9 doubles damage (Champions rules, Serebii-sourced abilities/status).

- **Bar (met):** within 5% of the Smogon damage calculator on 100% of 31 scenarios (`data/damage-validation.json`), gated in CI.
- **Role:** the exact leaf evaluator + rollout simulator everything else uses.

I1. Belief model (opponent set + next-move) — TO BUILD

- **Is:** given the observed battle so far, a **distribution over the opponent's hidden sets** (item/ability/spread/moves) and their **next move**, updated by a Bayesian filter on observed moves + damage, seeded by usage priors (our behaviour-clone).
- **Build:** start from the current clone (top-1 36% / top-3 72%); add (a) **set inference** (narrow the set as moves/items reveal — Champions is open-sheet-ish so this is tractable), (b) a small **move predictor** conditioned on state (active mons, HP, field), trained on replays.
- **Bar:** held-out **top-1/top-3 move-match + cross-entropy vs the clone baseline**, with CI (extend `engine/eval_policy.py`); set-inference measured by log-loss of revealed sets. Must beat the clone.

I2. Value/Policy net (the learned brain) — TO BUILD (biggest piece)

- **Is:** the Metamon lesson — a model trained by **imitation + offline RL** on our human replays (grows with the daily pull) + **self-play** trajectories, outputting a **value** $V(\text{state})$ and a **policy** $\pi(\text{move}|\text{state})$. This is what makes ALAKAZAM strong without heavy search.
- **Build:** Phase-2a imitation (behaviour cloning of winning play) → Phase-2b offline RL (advantage-weighted / CQL-style) → Phase-2c self-play fine-tune, **KL-anchored to the imitation policy (piKL)** so it stays human-realistic and unexploitable. Leaf value kept **small/fast (NNUE-style)** for later search.
- **Bar:** (a) action-value calibration + top-k on held-out human decisions; (b) **self-play / ladder win-rate vs the behaviour-clone and heuristic baselines**, with CIs — the honest "is it actually better at deciding" test. No match-winner-prediction claim.

I3. SLOWKING (team-preview Nash, outer game) — TO BUILD

- **Is:** at preview, the **bring-4 / lead-2 mixed strategy** and equilibrium value, by solving the matchup as a **matrix game** (regret matching / LP, `engine/slowking/nash.py`) over the **belief** of the opponent's sets — not a greedy best-response (that's what inverted MEDICHAM).
- **Bar:** exploitability of the returned strategy ($\downarrow$ is better); head-to-head self-play win-rate vs greedy bring. Team-rating (if shown) uses **Nash-averaging**, never scalar Elo.

I4. CHOMP (pivot + prove) — PARALLEL

- **Is:** bring/lead as **expected value over the belief** (xG-for-preview), grounded in validated damage; emits a **calibrated edge** (Kelly/calibration framing), not a win oracle.
- **Bar (the winnable test):** do CHOMP's recommended brings beat the human's actual **brings**, measured over many held-out games by realized result + a proper score with CI. This is the empirical proof CHOMP is worth it — distinct from (impossible) match prediction.

What we retire/reframe (not delete)

- **JOLTEON** → demoted to a fast **usage-prior / shortlister**; no win-% claim. (Optionally re-tried on a blade-chest/disc class purely as a descriptive meta-rating — low priority.)
- **MEDICHAM win%** → not shown as $P(\text{win-the-game})$; used internally as a **de-biased matchup value** and the rollout simulator. Its damage stays central.

- **DITTO** → **PSRO/double-oracle team-builder** (grow population → best-respond to meta-Nash → Nash-mixture), objective = coverage + validated damage + SLOWKING value, **not** the inverted win%.

Additional inputs the wider study calls for (do it all)

Beyond I0–I4, the literature (poker endgame solving, AlphaStar scouting, CICERO λ -anchoring, sports calibration, VGC-Bench PSRO) implies these. Added to the plan; each ships with metric + CI + baseline.

More input models

- **I5. Meta / matchup model** — (a) usage prior + **P(opponent archetype | what's revealed)** (pivot `engine/archetypes.py`); (b) **learned matchup matrix estimated from REAL game outcomes** (aggregate head-to-head archetype win-rates from the stored games, with Wilson CIs) — this replaces the *biased simulated* payoff matrix in SLOWKING / DITTO / non-transitivity, killing that GIGO at the source. *Bar*: held-out log-loss of the matchup predictions vs a usage baseline.
- **I6. Opponent-type / exploitability model** — infer opponent **skill** (rating + deviation from equilibrium play) and set the **piKL anchor strength λ** accordingly: play near-Nash vs strong opponents (safe/unexploitable), best-respond/exploit vs weak ones (CICERO's DiL-piKL, poker exploitative play, AlphaStar league range). *Bar*: exploit-rate gain vs fixed weak agents in self-play without added exploitability vs strong ones.
- **I7. Endgame exact solver** — when few mons remain ($\leq 2v2$, $1v1$), the game is small enough to **solve exactly** (retrograde / full matrix-game solve). Gives ground-truth leaf values and clean **training targets for the I2 value net** (the poker endgame-solving trick). *Bar*: exactness vs brute force on toy endgames.
- **I8. Self-play data engine + curation** — the fuel for I2 (Metamon used 5M human + **20M self-play**). Formalize `sim/` into a scalable self-play generator writing to the store schema, plus a **dedup / quality filter** on replays. *Bar*: dataset size + a quality audit (no leakage, balanced, deduped).
- **I-Gimmick. Battle-gimmick module (currently Mega) — design principle: model ONE gimmick at a time**, as a **swappable module keyed off** `data/regulations.json`, because each regulation has exactly one active gimmick (Champions Reg M-B = Mega; other regs = Tera or Z-Moves, inactive here). No need to ever carry more than one. For **Mega** specifically: the full forme transform (**stats + types + ability**, not just the ability we did), the **once-per-battle Mega-timing decision**, and a **belief over whether/when the opponent Megs. Mega-timing is genuinely strategic, not automatic** — most Pokémon Mega turn 1, but the model must recognize the +EV hold cases:
 - **speed**: keep a faster **base speed tier** to move first this turn (some Megs are slower, or you want the base speed now and the Mega bulk/power later);
 - **base-ability sequencing**: trigger a valuable **base ability first**, then Mega — e.g. Intimidate on entry, **re-set the weather** at the right moment with a base weather-setter, or **farm Moody boosts** (Scovillain runs Moody in base forme, stacking +2s before Mega-ing into Spicy Spray);
 - **information: don't reveal the Mega** (its typing/ability) until forced — an info play that ties into the scouting/information-value reward. When the format rotates, drop in a new gimmick module, not a rewrite. *Bar*: correct forme stats/types vs Serebii; does modeling the hold cases beat naive auto-Mega-turn-1 in ALAKAZAM self-play?

Supporting components (required, not standalone models)

- **Calibration layer** — temperature / isotonic on **every** probability we emit (the review's mandate; sports/betting calibration). One utility, applied everywhere, reliability-tested.
- **Information-gain (scouting) reward** — value moves that **reveal** the opponent's hidden set, added to ALAKAZAM's search reward (AlphaStar scouting).
- **Variance / risk estimate** — how much a line depends on rolls / crits / misses → an honest "**how coin-flippy is this**" readout alongside the EV (sports xG smoothing; poker variance).
- **Legality / format checker** — Champions dex + item/species clause, so DITTO/PSRO only generate **legal** teams.
- **Mechanics-coverage tracker** — % of observed moves/abilities the engine models correctly; a live **GIGO gauge** so we always know the simulator's blind spots.

Resolved open question

- **Terastallization: NOT active in Reg M-B** (Serebii/game8 + confirmed) — no Tera model. **Mega Evolution is the only gimmick**; complete the **I-Mega** model instead.

The capstone — ALAKAZAM (built last)

Input: a live position (both actives, HP, field, revealed info) + I1 belief + I2 value/policy + I3 strategic context. **Process:**

1. Form the **public belief state** (I1) over the opponent's hidden info.
2. **Policy proposal** from I2 (fast, KL-anchored to human play).
3. **Depth-limited search** (ReBeL/DeepStack-style) over the **validated damage engine**, solving **each simultaneous turn as a matrix game** (regret matching — kills strategy fusion + speed bias), leaves evaluated by the I2 value net. Reward includes **information-gain** (AlphaStar scouting) so it values forcing reveals.
4. **Output:** the **mixed move recommendation**, the **expected win-% delta** of each option (calibrated EV, not an oracle), and a plain-English **reason** (RAG-of-knowledge, PokéLLMon-style). This is KADABRA fully evolved.

Bar: decision-quality vs held-out human play + **self-play/ladder win-rate vs I2-alone and the behaviour-clone**, with CIs. Search kept only if it beats the no-search policy (Metamon showed it might not — measure honestly). Runs in a **Web Worker/WASM/backend**, never the main thread.

The end-game deliverable — PLAY ALAKAZAM AT A CHOSEN ELO

The capstone is not a number in a table. It is: **a human opens the site and plays a full Champions game against ALAKAZAM, with a difficulty dial set in Elo**. Everything above is the machinery that makes that possible; this is the thing the project is actually for.

Why an Elo dial is the right interface, and why it is more than a slider. A model that only ever plays as hard as it can is useless for training — you lose, learn nothing about *why*, and cannot practise a specific matchup against a specific standard of opponent. The literature is unambiguous that calibrated difficulty is a distinct problem from strength:

- **Maia** (McIlroy-Young et al., KDD 2020) trained *separate* networks per rating band on human chess games and showed each one predicts the moves of players at **that** rating far better than a strong engine does. Weakening a strong engine does **not** reproduce a 1200-rated player — it produces a 2800-rated player making occasional absurd blunders, which is a different and unconvincing thing.
- **AlphaStar** (Vinyals et al., *Nature* 2019) maintained a **league** of distinct agents rather than one optimum, precisely because a single self-play optimum is exploitable and unrepresentative.
- **Metamon** (Grigsby et al., 2025) is the direct precedent in Pokémon: policies trained on human ladder data at different skill strata behave differently, not merely worse.

So the dial must select **between policies trained to imitate a band**, not a single policy with noise injected. ABRA is already positioned for this: the ladder store carries a rating on most games, and MEW's policy layer is swappable (`--policy`), which is exactly the seam a band-conditioned policy plugs into.

Design.

1. **Band the ladder store by rating.** Measure, do not assume, how much data each band has — the statistical-power block already in `eval_harness.py` decides which bands are supportable. Bands with too little data are not offered rather than being offered dishonestly.
2. **Behaviour-clone one policy per band** (the BCSP recipe VGC-Bench found strongest: clone, then self-play from the clone). Bands share the state encoder and the value net; only the policy head differs.
3. **Calibrate the dial empirically.** The label "1500" must be *earned*: the agent set to 1500 plays a large sample against the other bands and against the behaviour clones, and its realised win rates must place it near 1500 on the standard Elo scale. A dial labelled with a rating it has not demonstrated is an asserted constant, which S-standards forbid.
4. **Ship it in the browser.** Inference for a policy of this size is small; search is the expensive part, so the Elo dial can also govern *search depth*, which is a legitimate strength lever on top of the band-conditioned policy — but never a substitute for it.

Acceptance bar. For each offered band: (a) realised Elo within a stated CI of the label, measured against the other bands; (b) move-agreement with held-out human games **at that band** exceeding agreement at every other band — the Maia test, and the one that distinguishes genuine imitation from a handicapped expert; (c) a full game playable in a browser without blocking the main thread.

Honest status. Not started. It depends on the value net (in progress), the belief model, and a self-play corpus large enough to train from. The rating data also has a known problem: this project has already measured that the better-rated player wins only about **52.4%** of ladder games, with a CI that includes 50 — so ladder Elo is a **weak** signal here, and band separation must be demonstrated rather than assumed. If bands cannot be distinguished, that is a finding and gets reported as one.

Build order (inputs first, capstone last)

1. **I0 damage engine** —  done.
2. **I1 belief model + I4 CHOMP-EV proof** (parallel; both extend `eval_policy` /backtest harnesses).
3. **I2 value/policy net** — imitation → offline RL → self-play, piKL-anchored. The heavy lift.
4. **I3 SLOWKING** preview-Nash on top of I1/I2.
5. **DITTO** → **PSRO** team-builder.
6. **ALAKAZAM** — assemble I0+I1+I2(+I3) with KL-anchored search. Ship the coach.

Each step: metric + CI + baseline, persisted to JSON, gated in CI. No capstone until its inputs clear their bars.